

ChimeraBoost: a whitepaper

Lightning-fast gradient boosting, near-CatBoost quality, all in Python.

Version 0.32.0 · September 2026 · Apache-2.0 · github.com/bbstats/chimeraboost

Abstract

ChimeraBoost is a gradient boosting library written entirely in Python. It depends on four packages (numpy, numba, scipy, and scikit-learn) and nothing else: no C++ extension, no GPU, and pandas is optional. On TabArena, the community tabular benchmark, its default configuration is the second strongest gradient boosting library on the board. Only CatBoost scores higher, and XGBoost and LightGBM defaults sit more than a hundred Elo points below. It reaches that quality while training in under half of CatBoost's median time, and it can do a few things the big libraries can't: quantile predictions that never cross, exact SHAP values without installing anything extra, and defaults good enough that most users never touch a hyperparameter.

The evidence comes first, then what it means in day-to-day use, then how the library works. Section 6 covers the things it is worse at.

1. What it is

ChimeraBoost is a scikit-learn-compatible GBDT with a deliberately small public API: seven names, three of which are estimators (ChimeraBoostRegressor, ChimeraBoostClassifier, and ChimeraBoostQuantileRegressor). It covers regression with squared error, absolute error, Huber, Poisson, Gamma, Tweedie, or a custom objective; binary and multiclass classification; and joint estimation of a full grid of conditional quantiles from a single booster.

The core design is CatBoost's oblivious tree: every node at a given depth shares one (feature, threshold) pair, so a tree of depth d is just d splits and a leaf index is a d -bit number. Most of the other ideas are borrowed too, and credited: ordered target statistics for categoricals, histogram split finding, minimum-variance sampling, quantized gradient histograms, vector leaves, linear leaves, exact TreeSHAP. The library's [attribution page](#) keeps track of what was borrowed, what was adapted, and the two contributions that are original.

One integer, `quality=1..5`, picks a point on the speed and accuracy trade-off; the default is 3. The library is roughly 11,700 lines across 13 modules, and 601 test functions cover it, including tests that pin the output of every performance optimization bit for bit.

2. The evidence

2.1 TabArena

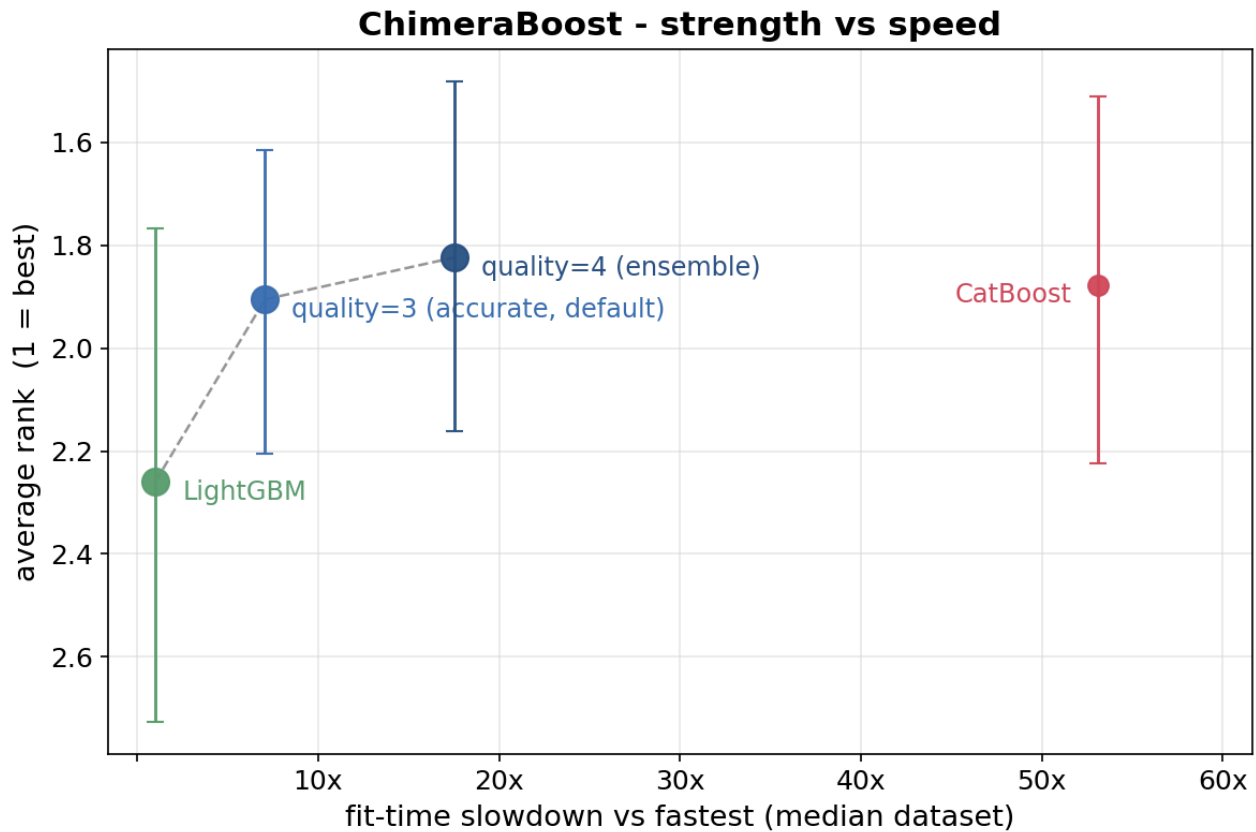
[TabArena](#) is a living leaderboard for tabular machine learning, maintained by its own authors (Erickson et al., NeurIPS 2025): 51 curated classification and regression datasets, every model run by the maintainers in a pipeline they optimize, ranked by Elo. ChimeraBoost is developed without ever tuning against it. TabArena results get reported and nothing more; no number from it, overall or per dataset, is allowed to influence a code change. Development decisions run on separate benchmark suites (section 5), which makes the table below a genuinely out-of-sample result.

As of September 2026, among gradient boosting libraries at default settings:

model (default config)	Elo	median train s/1K	median predict s/1K
CatBoost	1379	5.88	0.025
ChimeraBoost	1315	2.65	0.052
XGBoost	1214	1.94	0.123
EBM	1204	6.67	0.014
LightGBM	1187	1.96	0.142

There are two ways to read that table. On accuracy, ChimeraBoost's default is second among GBDTs, and the 101-point gap down to third place is larger than the 64-point gap up to first. On cost, it trains in about 45% of CatBoost's median time per thousand rows and predicts two to three times faster than XGBoost or LightGBM at their defaults, although CatBoost still predicts faster than any of them except EBM. ChimeraBoost's tuned-and-ensembled entry scores 1383, just above CatBoost's default, using a search space from June that predates several of the library's newer adaptive defaults. For context, the top of the overall board belongs to tabular foundation models around Elo 1790; those are pretrained neural networks in a different weight class of compute.

2.2 The public suite



The published chart runs on 22 independently audited public datasets that overlap with nothing the defaults were tuned on. In the most recent full run (July 2026, three seeds, 21 of 22 datasets scored):

model	avg rank	95% CI	median slowdown
ChimeraBoost quality=4 (ensemble)	1.73	1.46-1.99	16.4x
CatBoost	1.92	1.53-2.31	52.0x
ChimeraBoost default	1.97	1.67-2.28	7.2x
LightGBM	2.23	1.74-2.70	1.0x

Each ChimeraBoost row is ranked in its own three-way contest against CatBoost and LightGBM, so 2.0 is the middle of the field. The short version: the default lands within noise of CatBoost while spending less than a third of the fit time, and the `quality=4` ensemble tops the table while still costing less than a third of what CatBoost does.

3. What that buys you

3.1 Data goes in as it is

Categorical columns are handled natively by ordered target statistics, CatBoost's scheme, averaged over four permutations. You can address them by column index or by DataFrame name. Missing values get their own histogram bin at fit and at predict, numeric and categorical alike. There is no one-hot step, no imputation step, and no label encoder. pandas isn't even a dependency; DataFrames are read through their own to_numpy.

3.2 Mostly, you don't tune it

The library does a fair amount of tuning on its own. During a fit it auditions linear leaves against constant leaves, and automatic cross features against plain ones, racing them on a shared budget and keeping the winner. The learning rate adapts to dataset size, an idea we found by diffing CatBoost's resolved parameters across dataset sizes and noticing that the learning rate is the one default it moves. Early stopping is on by default, and the stopped model is refit on all rows by replaying its structure (section 4). What's left for you is one integer:

quality	profile	fit time (multiple of the fastest library)
1	fast	2.7x
2	balanced	5.3x
3	accurate (default)	6.9x
4	ensemble of 5	18.1x
5	ensemble of 8	26.0x

Every rung buys accuracy for its extra time, so none of them is a bad deal.

3.3 A full predictive distribution from one model

ChimeraBoostQuantileRegressor fits a whole predictive distribution, 19 quantile levels by default, in one booster: one tree structure per round, and a vector in every leaf with one entry per level. From that single fitted grid it can answer, at no extra cost: central intervals, the median, the mean, the CDF at any threshold, inverse-transform samples for downstream simulation, and `predict_thresh(X, t)`, the probability that the outcome exceeds a threshold, per row.

Three properties set it apart from stacking up per-level quantile models.

Predictions never cross. The 30% quantile is never returned above the 70%. Every row is sorted on the way out, and sorting a crossing quantile curve never increases pinball loss at any level (Chernozhukov, Fernández-Val and Galichon, 2010), so the guarantee costs nothing. Independently

fitted per-level models have no such property: across 36 real regression datasets, LightGBM's per-level boosters reverse 22% of adjacent level pairs on average and cross on every single dataset, and CatBoost's shared MultiQuantile head also crosses on every dataset. ChimeraBoost's crossing rate is exactly zero on all 36.

The intervals hold up, and can be guaranteed. The raw grid's nominal 80% and 90% intervals deliver 77% and 87% observed coverage, closer to nominal than LightGBM per-level (72% and 83%) or CatBoost MultiQuantile (73% and 83%). Setting `conformalize=True` upgrades that to distribution-free marginal coverage, measured within 2.7 points of nominal at $n = 10,000$. An out-of-fold study on seven real datasets (five folds each, September 2026) backs this up: `predict_thresh` probabilities land within an expected calibration error of 0.03 to 0.05 of reality, the crossing rate is zero everywhere, the raw grid's slight narrowness shows up only in the outermost tail cells, and conformalization repairs exactly that, bringing 90% coverage to between 0.902 and 0.905 on the three worst datasets.

You can explain the width. SHAP attributions come per level, and also for the width of an interval: which features make this particular row's prediction uncertain, rather than simply higher or lower. A stack of independent per-level models can't produce that decomposition, because it needs both edges of the interval to come from one model.

Head to head on those 36 datasets, with every arm early-stopping on the same split: against 19 independent LightGBM quantile boosters, CRPS is roughly a tie (23 wins, 13 losses), while the interval score, the proper rule that prices coverage and width together, goes 31 to 5 in ChimeraBoost's favor at two thirds of the fit time. CatBoost's MultiQuantile beats us on CRPS; section 6 has the details.

3.4 Explanations and calibrated probabilities, built in

`shap_values` computes exact interventional TreeSHAP. No shap package, no sampling approximation, and roughly 3 ms per row at depth 6. Oblivious trees are why exactness is affordable: a depth-6 tree involves at most six features, so all 64 coalitions can simply be enumerated. Attributions stay exact through per-leaf linear models, and work for multiclass and quantile heads too. Classifier probabilities are temperature-scaled on the early-stopping fold; the scaling is monotone, so accuracy and AUC don't move while calibration improves, and the fitted `temperature_` is there to inspect.

3.5 Deployment is four packages

The wheel depends on numpy, numba, scikit-learn, and scipy, and nothing else. Predict latency at TabArena's scale is 0.05 s per thousand rows. The one real tax is numba's compiler: the first fit on a fresh install spends about ten seconds compiling (0.8 s once warm, 0.27 s steady state), and upgrading the package pays it again. There is a `chimeraboost-warmup` command and an environment variable to precompile in the background, and the [deployment guide](#) walks through the details.

4. How it works

Oblivious trees provide the speed and much of the regularization. Because a leaf is a bit pattern of d shared comparisons, prediction is a handful of comparisons and one table lookup per tree, executed in a single numba kernel parallelized over samples, so each row walks the entire forest while its features stay in cache. The constraint that one (feature, threshold) serves a whole depth also acts as a strong prior against overfitting. On very clean, high-signal data the same constraint becomes a handicap against leaf-wise learners; section 6 comes back to this.

Histograms, quantized. Split finding runs on binned features (128 bins by default, with missing values binned separately), and gradients and Hessians are quantized to 15-bit integers, following Shi, Ke et al. (NeurIPS 2022). Integer histograms fit 20 to 25% faster at the same measured accuracy. Leaf values are always computed from exact float gradients, and results are deterministic for a fixed seed.

Vector leaves give multiclass one tree per round instead of one per class. Fits run 2.5x faster, and the Brier score actually improves 5% at equal F1. The same machinery gives the quantile head its per-leaf vector.

Replay refit is one of the library's two original contributions. Early stopping needs a validation fold, so the stopped model has seen only part of the data, and a from-scratch refit on all rows used to cost 37 to 49% of every default fit. Instead, ChimeraBoost replays the winning structure, the recorded splits round by round, against gradients computed on all rows, refitting only the leaf values. Growing trees is 83 to 85% of a fit, and replay skips it entirely. Total fit time dropped 34.8% on the 59-dataset decision suite, faster on 58 of 59, with accuracy a wash across four independent suites.

A gradient-matrix-free quantile head is the other. Each row's K -channel pinball gradient collapses to a single projected value in one pass, so the $(n \times K)$ gradient matrix is never materialized and a 19-level head costs about what a 1-level head costs per round, landing within 7% of the exact summed-gain answer. The projection matters: naively summing gradients across a symmetric grid is blind to spread, because the pushes from level τ and level $1 - \tau$ cancel exactly for an interval that is centered but too narrow.

Bagging without replacement. `n_ensembles` fits members on 80% subsamples drawn without replacement, which measured better than the classic bootstrap on both accuracy and fit time. Members fit in parallel worker processes sharing one thread budget. `refit_members=True` replays each member on its full data, and five refit members beat eight plain ones on both accuracy and speed.

5. How the numbers are kept honest

Benchmark numbers are only as trustworthy as the process behind them, so it is worth describing the process. Development uses five suites, each with a single job: a synthetic screen to check whether an idea does anything at all; a decision suite (Grinsztajn et al. 2022 plus a frozen high-cardinality set)

where changes ship or die by per-stratum sign tests; a tuning suite (PMLB), the only data hyperparameters are allowed to see; the 22-dataset audited public suite for published evidence; and TabArena, which is never tuned against. A suite you tune on can't also tell you whether you've overfit it, which is why these stay separate, and why the leaderboard quoted above never feeds back into the code. Datasets enter a suite based on data properties alone, things like row counts, cardinality, and missingness, never based on results.

The same standard applies to old claims. When the quantile head's early speed numbers (3 to 6x versus LightGBM) turned out to be an artifact of fixed-round fits on synthetic data, the documentation was corrected to the real-data figure of 1.5x, with a note on the page explaining what happened.

6. Limitations

- **CatBoost's MultiQuantile is sharper on CRPS.** It wins 29 of 36 datasets against our quantile head, and that gap is real. It also costs a median 8.3x our fit time and its intervals are much worse calibrated (its worst coverage error is 0.64 against a nominal 0.90, ours is 0.10), which is why the interval score still favors us 25 to 1.
- But if CRPS is the only thing you care about and fit time is no object, CatBoost wins it.
- **No GPU**, by design. Staying pure Python is the point of the project.
- **Compiler cold start.** Expect about ten seconds of one-time compilation per fresh install or upgrade. The warmup tooling softens this without eliminating it.
- **The oblivious tax.** On clean, high-signal data, leaf-wise learners can carve sharper structure than symmetric trees can. The regularization that wins on noisy real-world tabular data works against you on nearly deterministic problems.
- **Training is not the fastest.** LightGBM and XGBoost defaults train in about 75% of our time on TabArena's hardware. ChimeraBoost's case is the quality you get for the compute you spend; if minimum training time is what matters, those two are quicker.
- **The API is 0.x beta.** Names have been stable in practice but aren't guaranteed yet.

7. Getting started

```
pip install chimeraboost
```

```
from chimeraboost import ChimeraBoostClassifier, ChimeraBoostQuantileRegressor

clf = ChimeraBoostClassifier()                # or quality=4 for the ensemble
clf.fit(X_train, y_train, cat_features=["city"]) # categoricals by name, NaNs fine
proba = clf.predict_proba(X_test)             # temperature-calibrated

qr = ChimeraBoostQuantileRegressor(conformalize=True)
qr.fit(X_train, y_num, cat_features=["city"])
lo, hi = qr.predict(X_test, kind="interval", alpha=0.1).T # guaranteed 90% band
p_over = qr.predict_thresh(X_test, 100.0)                # P(y > 100) per row
```

Documentation lives at bbstats.github.io/chimeraboost. The benchmark harness, raw results, and every study cited here are in the [repository](#) under benchmarks/.

Sources and further reading

The TabArena figures are the official leaderboard as of 2026-09-01, recorded in benchmarks/tabarena/LEADERBOARD_SNAPSHOT.md. The public-suite figures come from the audited run of 2026-07-27, recorded in benchmarks/PUBLIC_PLAN.md. The quantile comparisons come from benchmarks/quantile_suite.py over 36 Grinsztajn regression datasets with three seeds ([full tables](#)), and the out-of-fold calibration study is benchmarks/quantile_oof_calibration.py (2026-09-01). Replay refit and the quantile split search are written up in [the attribution page](#), along with per-feature citations for the research ChimeraBoost builds on: CatBoost, XGBoost, LightGBM, minimum-variance sampling, quantized GBDT training, SketchBoost and GBDT-MO, linear-leaf trees, TreeSHAP, and OpenFE.